

# ARYAN SHARMA

## Low-Latency & High-Performance Engineer

Ghaziabad, India · +91 93154 65182 · [aryansharma24112003@gmail.com](mailto:aryansharma24112003@gmail.com)

[github.com/Aryan-Protein-Vala](https://github.com/Aryan-Protein-Vala) · [linkedin.com/in/aryannnn](https://linkedin.com/in/aryannnn) · [aryannnn-portfolio.vercel.app](https://aryannnn-portfolio.vercel.app)

---

### SUMMARY

Performance engineer who profiles, root-causes, and fixes at the millisecond level: a voice RAG system answering in 0.8 ms P50 round-trips (96% latency reduction from one TCP\_NODELAY fix), a zero-copy file engine saturating Gigabit-class links (71 MB/s measured), and a 25M-cell real-time engine with sub-millisecond state fan-out. I measure everything — per-stage telemetry is a feature, not a debugging afterthought.

---

### TECHNICAL SKILLS

- Latency Engineering: P50/P70/P100 telemetry · per-stage pipeline benchmarking · TCP\_NODELAY/Nagle/delayed-ACK profiling · zero-copy I/O (mmap) · in-RAM vector search · LRU caching · speculative/pre-fetch execution
- Throughput: raw TCP multiplexing (8 MB socket buffers, 16 MB parallel chunks) · hardware SHA-256 (SHA-NI) · goroutine fan-out · MPSC channel pools · sub-millisecond Pub/Sub fan-out
- Profiling & Verification: scripted audit harnesses · in-process benchmarks as regression gates · deterministic sandboxes · 96% latency reduction case study (44 ms → 1.5 ms)
- Languages: Rust · Go · Python · C (background) · Linux internals (socket buffers, mmap) · Docker

---

### EXPERIENCE

#### Cinntra Infotech — Systems Engineer · 2026 – Present

- Built CIRA, a RAG agent translating natural-language requests into audited SAP HANA queries and write operations, giving non-technical users direct access to enterprise data
- Built the point-of-sale system backend: transaction recording, merchant reconciliation, and offline synchronization

#### Niswa Engineering — Technical Consultant & Automation Engineer · 2024 – 2026

- Built asynchronous AI agents (TypeScript) that automated 80%+ of a content generation and scheduling pipeline
- Built the company website in Astro (95+ Google Lighthouse score); set up cloud infrastructure, API integrations, and event-driven automation workflows

#### Sudarshana Foundation — Full-Stack & Automation Engineer · Contract

- Designed and built the organization's first centralized web platform, digitizing paper-based workflows into software
- Built database systems handling 5,000+ monthly transactions

---

### PROJECTS

#### VYU — 0.8 ms P50 Voice RAG System

- Delivered a 50–100 ms latency requirement at ~1% of budget: P50 0.8 ms, P70 1.5 ms, worst-case 1.6 ms end-to-end WebSocket round-trips, verified by a live 10-query audit harness.
- Root-caused the real bottleneck: uvicorn's websockets path never set TCP\_NODELAY — Nagle plus the peer's 40 ms delayed ACK added ~42 ms to every round trip. Fixed in the server (enable TCP\_NODELAY for WS sockets) and verified 44 ms → 1.5 ms (96% reduction).
- Built the per-stage telemetry that keeps it honest: guardrail 0.005 ms → 512-dim TF-IDF embed 0.51 ms → FAISS IndexFlatIP 0.017 ms → O(1) parent-chunk lookup 0.009 ms → grounding 0.005 ms (0.67 ms P50 retrieval), with an in-process benchmark as the standing regression gate.
- Engineered the speculative path: retrieval starts on live speech partials, so candidates are in RAM before the user finishes the sentence; LRU cache serves repeat/similar queries in <0.2 ms; barge-in cancels in-flight generation instantly.
- Tech: Python, FastAPI, Uvicorn, FAISS, WebSockets — [github.com/Aryan-Protein-Vala/vayu-voice-rag](https://github.com/Aryan-Protein-Vala/vayu-voice-rag)

#### Aether — Saturated-NIC Zero-Copy Transfer Engine (Go)

- Eliminated the three classic bottlenecks: GC (zero heap allocations via mmap pre-mapping), disk I/O bounce (io.ReadFull from NIC buffers directly into mapped file pointers), and software crypto (hardware SHA-256 on 10 parallel goroutines).
- Custom raw TCP multiplexer (persistent TCP\_NODELAY sockets, 8 MB OS buffers, 16 MB parallel chunks, [len][frame] protocol) — modeled 110–115 MB/s on Gigabit, 700–1,100 MB/s on 10G; 71 MB/s peak measured sync throughput.
- Tech: Go, syscall/mmap, raw TCP, QUIC, mDNS — [github.com/Aryan-Protein-Vala/AetherNet](https://github.com/Aryan-Protein-Vala/AetherNet)

#### CLAIM/05 — Sub-Millisecond Multiplayer State Fan-Out

- Built the Go WebSocket engine (hub pattern, concurrent channel hubs) that syncs a 25,000,000-cell live battlefield: Redis SETNX atomic cell locks + Pub/Sub delta fan-out reaching all connected clients in sub-millisecond, with reload-safe reconnection and a GitHub Actions keep-alive holding the 24/7 engine at zero cold starts.
- Tech: Go, Gorilla WebSockets, Redis Pub/Sub — [github.com/Aryan-Protein-Vala/Grid-Game](https://github.com/Aryan-Protein-Vala/Grid-Game) · live: [multiplayer-grid-game-theta.vercel.app](https://multiplayer-grid-game-theta.vercel.app)

#### CORTEX — O(1) Memory Injection (Rust)

- Built the memory injection path to be flat-cost: Qdrant vector pre-filter into the SurrealDB graph, decay-driven pruning, and a hyper-dense context packet returned in O(1) at the API layer — memory size grows without token cost growing.
- Tech: Rust, SurrealDB, Qdrant, tonic/gRPC — [github.com/Aryan-Protein-Vala/CORTEX](https://github.com/Aryan-Protein-Vala/CORTEX)

---

### CERTIFICATIONS

- Frontend Development Certification — Sheryians Coding School
- Data Science & AI/ML Bootcamp Certification — Udemy